

PROGRESS REPORT — PHASE 1

Integrated Digital Business Platform

Authentication & Access Control System

ORGANIZATION

S.N. Polymers

REPORT DATE

4 June 2026

PREPARED BY

Development Team — Intern Batch 2026

PROJECT PHASE

Phase 1 of 4 — Complete

FRONTEND DEPLOYMENT

sn-polymers.vercel.app

BACKEND DEPLOYMENT

snpolymers.onrender.com

Progress Report — Integrated Digital Business Platform (IDBP)

Phase 1: Authentication & Access Control System

Organization: S.N. Polymers **Report Prepared By:** Development Team — Intern Batch 2026 **Report Date:** 4 June 2026 **Project Repository:** github.com/Zenoguy/SNPolymers **Reporting Period:** 2 June 2026 – 4 June 2026

Executive Summary

The development team has successfully completed **Phase 1** of the Integrated Digital Business Platform (IDBP) for S.N. Polymers. This phase focused on building a secure, production-ready authentication and access control system that will serve as the foundational security layer for all future ERP modules.

The system is **fully built and deployed** across two cloud platforms. The frontend portal is live on Vercel and the backend REST API is hosted on Render, both connected to a Supabase (PostgreSQL) database. The platform implements OTP-based authentication via WhatsApp (Twilio WABA), JWT session management, role-based access control, automated administrative notifications, and a complete audit trail of all user sessions.

1. Project Objectives

The objectives defined for Phase 1 were as follows:

#	Objective	Status
1	Build a secure internal portal accessible only to whitelisted mobile numbers	✅ Complete
2	Implement OTP-based login via WhatsApp (Twilio WABA)	✅ Complete
3	Establish JWT-based session management with httpOnly cookie storage	✅ Complete
4	Provide an Admin Panel for user whitelist management (CRUD)	✅ Complete
5	Implement automated email notifications to Administrator on every login/logout	✅ Complete
6	Build a full session audit log for corporate compliance traceability	✅ Complete
7	Deploy the system to cloud infrastructure (Vercel + Render + Supabase)	✅ Complete

2. Technology Stack

2.1 Frontend

Technology	Version	Purpose
-----	-----	-----
React	19.2.6	UI component framework
Vite	8.0.12	Development bundler & build tool
React Router DOM	6.23.1	Client-side routing & navigation
Tailwind CSS	3.4.4	Utility-first styling with PostCSS
Axios	1.7.2	HTTP client with interceptors

2.2 Backend

Technology	Version	Purpose
-----	-----	-----
Node.js	≥ 18.0.0	JavaScript runtime
Express.js	4.19.2	REST API framework
Supabase JS Client	2.43.4	Database & storage client
jsonwebtoken	9.0.2	JWT token signing & verification
bcrypt	6.0.0	Cryptographic OTP hashing
Twilio	5.1.0	WhatsApp OTP delivery via WABA
Nodemailer	8.0.10	Gmail SMTP email notifications
express-rate-limit	7.3.1	Request rate limiting middleware
cookie-parser	1.4.6	HTTP cookie parsing
uuid	14.0.0	JWT JTI (unique token ID) generation

2.3 Database & Hosting

Service	Purpose
-----	-----
Supabase (PostgreSQL)	Primary database with Row Level Security
Vercel	Frontend hosting (CDN-backed, global)
Render	Backend API hosting

3. System Architecture



4. Database Design

Three tables were designed and created in Supabase PostgreSQL with Row Level Security (RLS) enabled on all:

4.1 `authorised_users` — Whitelist Registry

<code>id</code>	<code>UUID</code>	Primary Key (auto-generated)
<code>mobile_number</code>	<code>VARCHAR(15)</code>	Unique, NOT NULL – E.164 format
<code>display_name</code>	<code>VARCHAR(100)</code>	
<code>role</code>	<code>VARCHAR(50)</code>	Default: 'staff' values: 'staff', 'admin'
<code>permissions</code>	<code>JSONB</code>	Default: {} – extensible for future modules
<code>created_at</code>	<code>TIMESTAMPTZ</code>	Auto-set on creation
<code>is_active</code>	<code>BOOLEAN</code>	Default: true – controls access without deletion

4.2 `otp_requests` — OTP Audit Table

id	UUID	Primary Key
mobile_number	VARCHAR(15)	NOT NULL
otp_hash	TEXT	bcrypt hash of 6-digit OTP (never stored in plaintext)
expires_at	TIMESTAMPTZ	OTP expiry timestamp (5 minutes from creation)
is_used	BOOLEAN	Default: false — marked true after successful use
attempts	INT	Default: 0 — incremented on wrong entry (max: 3)
created_at	TIMESTAMPTZ	

4.3 `sessions` — Live Session & Audit Trail

id	UUID	Primary Key
user_id	UUID	Foreign Key → <code>authorised_users.id</code>
login_at	TIMESTAMPTZ	Auto-set on creation
logout_at	TIMESTAMPTZ	Populated on logout
duration_seconds	INT	Computed on logout (<code>logout_at - login_at</code>)
ip_address	INET	Captured from request
user_agent	TEXT	Browser/client info
module	VARCHAR(50)	Default: 'office'
jwt_jti	VARCHAR(100)	Unique JWT ID for blacklisting
is_active	BOOLEAN	Default: true — set to false on logout

5. Completed Deliverables

5.1 Backend — REST API

5.1.1 Application Entry Point (`backend/src/app.js`)

- Express.js server configured with CORS, cookie-parser, and JSON body parser
- Dynamic CORS origin control via `FRONTEND_URL` environment variable
- `trust proxy` set to `1` for correct client IP detection behind Render's reverse proxy
- Global 404 and error handler middleware
- `/health` endpoint for uptime monitoring
- Runs on port `5000` by default

5.1.2 Authentication Controller (`backend/src/controllers/auth.controller.js`)

Four endpoints are fully implemented:

Function	Endpoint	Description
----------	----------	-------------

-----	-----	-----
-------	-------	-------

<code>requestOtp</code>	<code>POST /api/v1/auth/request-otp</code>	Validates mobile against whitelist, generates OTP, sends to WhatsApp
-------------------------	--	--

<code>verifyOtpCode</code>	<code>POST /api/v1/auth/verify-otp</code>	Verifies OTP, creates DB session, issues JWT in httpOnly cookie
----------------------------	---	---

<code>logout</code>	<code>POST /api/v1/auth/logout</code>	Closes session, clears cookie, notifies admin asynchronously
---------------------	---------------------------------------	--

<code>getMe</code>	<code>GET /api/v1/auth/me</code>	Returns current authenticated user profile
--------------------	----------------------------------	--

5.1.3 Admin Controller (`backend/src/controllers/admin.controller.js`)

Five admin endpoints fully implemented:

Function	Endpoint	Description
----------	----------	-------------

-----	-----	-----
-------	-------	-------

<code>getUsers</code>	<code>GET /api/v1/auth/admin/users</code>	Lists all whitelisted users with session counts & last login
-----------------------	---	--

<code>addUser</code>	<code>POST /api/v1/auth/admin/users</code>	Adds new user to whitelist with duplicate detection
----------------------	--	---

<code>updateUser</code>	<code>PATCH /api/v1/auth/admin/users/:id</code>	Updates user info or deactivates; auto-invalidates active sessions
-------------------------	---	--

<code>removeUser</code>	<code>DELETE /api/v1/auth/admin/users/:id</code>	Permanently removes user; invalidates all sessions first
-------------------------	--	--

<code>getSessions</code>	<code>GET /api/v1/auth/admin/sessions</code>	Full audit log with optional <code>userId</code> , <code>dateFrom</code> , <code>dateTo</code> filters
--------------------------	--	--

5.1.4 OTP Service (`backend/src/services/otp.service.js`)

- `generateOtp()` — Cryptographically secure 6-digit OTP using `crypto.randomInt()`
- `hashOtp(otp)` — bcrypt hash with 10 salt rounds; plaintext is never persisted
- `storeOtp(mobile, hash)` — Inserts OTP record with 5-minute expiry timestamp

- `verifyOtp(mobile, rawOtp)` — Retrieves latest unused OTP, checks expiry, compares bcrypt hash, tracks & limits attempts (max 3)

5.1.5 Session Service (`backend/src/services/session.service.js`)

- `generateToken(user)` — Signs JWT with HS256, unique `jti` UUID per token, 24h expiry
- `createSession(...)` — Inserts session row capturing `user_id`, `jti`, `ip_address`, `user_agent`
- `closeSession(jti)` — Updates session with `logout_at`, `is_active=false`, and computed `duration_seconds`
- `formatDuration(seconds)` — Formats session duration as `HH:MM:SS`

5.1.6 WhatsApp OTP Service (`backend/src/services/whatsapp.service.js`)

- Twilio client initialized from environment variables
- `sendOtp(mobile, otp)` — Dispatches WhatsApp template message via WABA
- **Development fallback:** When `NODE_ENV=development` or credentials are absent, OTPs are printed to console — no real message is sent

5.1.7 Email Notification Service (`backend/src/services/email.service.js`)

- Nodemailer transport configured with Gmail SMTP + App Password
- `notifyAdminLogin(...)` — Sends HTML email with user details (name, mobile, IP, user agent, timestamp)
- `notifyAdminLogout(...)` — Sends HTML email with session duration and logout timestamp
- Both run **asynchronously** via fire-and-forget callback — never blocks the HTTP response
- Console-logging fallback when SMTP credentials are not configured

5.1.8 Middleware

| File | Purpose |

|-----|-----|

| `verifyJwt.js` | Extracts JWT from httpOnly cookie → verifies signature → checks JTI against DB sessions table (blacklist check) → validates user is still active → attaches `req.user` and `req.jti` |

| `requireAdmin.js` | Checks `req.user.role === 'admin'` ; returns 403 if not |

| `rateLimiter.js` | Limits OTP requests to 3 per mobile per 15 minutes using `express-rate-limit` |

5.2 Frontend — React Dashboard Portal

5.2.1 Routing & App Structure (`frontend/src/App.jsx`)

Full React Router DOM v6 setup with nested protected routes:

| Route | Access | Component |

|-----|-----|-----|

| `/` | Public | `Home` — Corporate landing page |

| `/login` | Public | `Login` — Mobile number entry |

| `/verify-otp` | Public | `OtpVerify` — OTP entry screen |

| `/dashboard` | Staff + Admin | `Dashboard` — Internal portal hub |

| `/admin` | Admin only | `AdminPanel` — User whitelist management |

| `/admin/sessions` | Admin only | `AuditLog` — Session history & filters |

5.2.2 Frontend Pages

`Home.jsx` — Corporate public landing page featuring:

- Sticky header with S.N. Polymers branding and dynamic login/dashboard navigation
- Hero section with gradient typography describing the IDBP
- Feature division highlights (Manufacturing & Government Infrastructure)
- "Official Corporate Gateway" badge with amber accent palette

`Login.jsx` — Mobile number entry screen:

- `+91` prefix selector with 10-digit input validation
- Submit triggers `POST /request-otp` and navigates to OTP screen on success
- Error state display for non-whitelisted numbers

`OtpVerify.jsx` — OTP verification screen:

- 6-box individual digit OTP input with auto-advance focus
- 5-minute countdown timer
- Resend OTP button with 30-second cooldown
- Submits to `POST /verify-otp`, navigates to dashboard on success

`Dashboard.jsx` — Internal staff dashboard:

- Protected; accessible only to authenticated staff and admin users
- Shows user information, navigation links, and session state
- Logout functionality that calls `POST /logout` and clears auth state

`AdminPanel.jsx` — Administrative user management panel:

- Full CRUD table of all authorised users showing name, mobile, role, status, session count, and last login
- "Add User" modal with input validation
- Deactivate and Delete actions per row with confirmation prompts
- Calls all admin user endpoints

`AuditLog.jsx` — Session audit log view:

- Paginated session history table
- Filter controls: by user, date range (from/to)
- Displays login time, logout time, duration, IP address, and user agent
- Calls `GET /admin/sessions` with dynamic query parameters

5.2.3 Shared Components

| Component | Purpose |

|-----|-----|

| `AuthContext.jsx` | React Context providing `user` state and `setUser` to entire application |

| `ProtectedRoute.jsx` | Calls `GET /me` on mount; redirects unauthenticated users to `/login` ; checks `allowedRoles` prop for role-based access |

6. Complete API Reference

| Method | Endpoint | Auth Required | Description |

|-----|-----|:-----|:-----|

| `GET` | `/health` | None | Server health check / uptime monitoring |

| `POST` | `/api/v1/auth/request-otp` | None | Whitelist check → generate OTP → dispatch via WhatsApp |

| `POST` | `/api/v1/auth/verify-otp` | None | Verify OTP → create session → issue JWT in httpOnly cookie |

| `GET` | `/api/v1/auth/me` | JWT (Staff/Admin) | Returns current user's profile and role |

| `POST` | `/api/v1/auth/logout` | JWT (Staff/Admin) | Closes session → clears cookie → notifies admin |

| `GET` | `/api/v1/auth/admin/users` | JWT (Admin) | List all whitelisted users with session stats |

| `POST` | `/api/v1/auth/admin/users` | JWT (Admin) | Add mobile number to whitelist |

| `PATCH` | `/api/v1/auth/admin/users/:id` | JWT (Admin) | Update user details or deactivate account |

| `DELETE` | `/api/v1/auth/admin/users/:id` | JWT (Admin) | Remove user from whitelist + invalidate sessions |

| `GET` | `/api/v1/auth/admin/sessions` | JWT (Admin) | Full session audit log with filter support |

7. Security Implementation

All six key security concerns from the original specification have been addressed:

| Security Concern | Mitigation Implemented |

|-----|-----|

| **OTP brute-force** | Maximum 3 verification attempts per OTP; fails gracefully with remaining attempts count |

| **OTP farming / flooding** | Rate limiter middleware: max 3 OTP requests per mobile number per 15 minutes |

| **OTP plaintext storage** | OTPs are bcrypt-hashed (10 rounds) immediately; plaintext is never written to the database |

| **JWT token theft** | JWTs stored exclusively in `httpOnly` , `Secure` , `SameSite` cookies — inaccessible to JavaScript |

| **Token reuse after logout** | Every protected request checks the JWT's `jti` against the `sessions` table; inactive sessions are rejected immediately |

| **Unauthorized access** | Row Level Security (RLS) enabled on Supabase tables; admin role verification is server-side only; CORS restricted to known frontend origin |

8. Deployment

The system is fully deployed and operational:

| Service | Platform | URL |

|-----|-----|-----|

| Frontend Portal | Vercel | <https://sn-polymers.vercel.app/> |

| Backend REST API | Render | <https://snpolymers.onrender.com> |

| API Health Check | Render | <https://snpolymers.onrender.com/health> |

| Database | Supabase (PostgreSQL) | Managed cloud instance |

Deployment Architecture Notes

- Frontend is served via Vercel's global CDN with zero-downtime deployments
 - Backend connects to Render via the `VITE_API_URL` environment variable set at build time
 - All secrets (JWT secret, Twilio credentials, SMTP password, Supabase service role key) are stored exclusively in platform environment variables — never in source code
 - CORS is locked to the Vercel frontend URL in production
-

9. Environment Configuration

The following environment variables are required for the backend (`backend/.env`):

Application

PORT=5000

NODE_ENV=production

FRONTEND_URL=https://sn-polymers.vercel.app

Supabase

SUPABASE_URL=<project_url>

SUPABASE_SERVICE_ROLE_KEY=<service_role_key>

JWT

JWT_SECRET=<min_256_bit_random_secret>

JWT_EXPIRY=24h

Twilio WhatsApp OTP

TWILIO_ACCOUNT_SID=<account_sid>

TWILIO_AUTH_TOKEN=<auth_token>

TWILIO_WHATSAPP_FROM=whatsapp:+14155238886

Nodemailer SMTP

EMAIL_USER=<sender_email>

EMAIL_APP_PASSWORD=<gmail_app_password>

ADMIN_EMAIL=<administrator_email>

10. File Structure Summary

```
SNPolymers/
├─ backend/

|   └─ src/

|       └─ app.js                ← Express server entry point

|       └─ controllers/

|           └─ auth.controller.js    ← OTP request, verify, logout, /me

|           └─ admin.controller.js   ← User CRUD + session audit log

|           └─ db/

|               └─ supabase.js       ← Supabase client singleton

|               └─ middleware/

|                   └─ verifyJwt.js   ← JWT + JTI session validation

|                   └─ requireAdmin.js ← Admin role guard

|                   └─ rateLimiter.js ← OTP request rate limiter

|                   └─ routes/

|                       └─ auth.routes.js    ← /api/v1/auth/*

|                       └─ admin.routes.js   ← /api/v1/auth/admin/*

|                       └─ services/

|                           └─ otp.service.js    ← OTP generation, hashing, storage, verification

|                           └─ whatsapp.service.js ← Twilio WABA OTP dispatch

|                           └─ email.service.js  ← Login/Logout admin email alerts
```

```
| | └─ session.service.js    ← JWT generation, session create/close

| └─ .env.example            ← Environment variable template

| └─ package.json

|

└─ frontend/

    └─ src/

        | └─ App.jsx          ← Route definitions & auth context wrapping

        | └─ main.jsx          ← React DOM entry point

        | └─ api/

            | └─ authApi.js    ← Axios API client

            | └─ components/

                | └─ AuthContext.jsx    ← Global auth state provider

                | └─ ProtectedRoute.jsx ← Role-based route guard

                | └─ pages/

                    | └─ Home.jsx        ← Public corporate landing page

                    | └─ Login.jsx       ← Mobile number entry

                    | └─ OtpVerify.jsx   ← OTP verification screen

                    | └─ Dashboard.jsx   ← Staff/Admin internal hub

                    | └─ admin/

                        | └─ AdminPanel.jsx ← User whitelist management

                        | └─ AuditLog.jsx   ← Session history & filters
```

```
└─ index.html

└─ tailwind.config.js

└─ vite.config.js

└─ package.json
```

11. Current Status

⚠ IMPORTANT

Phase 1 is functionally complete.
All planned features have been implemented and the application is deployed to production.

Component	Completion
----- :-----	
Backend API — Core Authentication	100%
Backend API — Admin Management	100%
Backend API — Session Audit	100%
Security Middleware (JWT, Rate Limit, Admin Guard)	100%
OTP Service (generate, hash, store, verify)	100%
WhatsApp Delivery (Twilio WABA)	100%
Email Notification Service	100%
Frontend — Public Landing Page	100%
Frontend — Login & OTP Screens	100%
Frontend — Internal Dashboard	100%
Frontend — Admin Panel (User CRUD)	100%
Frontend — Audit Log Viewer	100%
Database Schema (Supabase / PostgreSQL)	100%
Cloud Deployment (Vercel + Render)	100%

12. Pending Items / Known Limitations

#	Item	Notes
---	------	-------

|---|-----|-----|

| 1 | **Twilio WABA credentials** | WhatsApp OTP requires an approved Twilio WABA account. The system falls back to console logging during development until credentials are provisioned. |

| 2 | **Admin Route Guard on Frontend** | Currently `ProtectedRoute` handles both staff and admin differentiation. A dedicated `AdminRoute` wrapper component was part of the plan and may be formalized as a separate component. |

| 3 | **Automated Test Suite** | No automated unit or integration tests have been written yet. Endpoint testing was conducted manually via browser and Postman. |

| 4 | **OTP SMS Fallback** | An SMS fallback via Twilio SMS (if WABA is unavailable) is not yet implemented. |

13. Phase 2 — Planned Scope (Future Work)

Phase 1 establishes the security gateway. Upcoming development phases will add domain-specific ERP functionality on top of the authenticated foundation:

- **Manufacturing Module** — Chemical formulation tracking, raw materials procurement, batch control, and internal stock auditing
- **Government Projects Module** — Tender tracking, logistics dispatching reports, work order scheduling, and municipal compliance management
- **Logistics Module** — Dispatch records, delivery routing, and vendor management
- **Reporting & Analytics** — Dashboard KPIs, business intelligence views, and exportable reports

14. References

| Resource | URL |

|-----|-----|

| Live Frontend | <https://sn-polymers.vercel.app/> |

| Live Backend API | <https://snpolymers.onrender.com> |

| Health Check | <https://snpolymers.onrender.com/health> |

| GitHub Repository | <https://github.com/Zenoguy/SNPolymers> |

| Phase 1 Documentation (PDF) | [IDBP_Phase1_Documentation.pdf](#) |

This document was prepared by the Development Team — Intern Batch 2026 for S.N. Polymers. Report Date: 4 June 2026 | IDBP Phase 1 — Authentication & Access Control